MLX Llama 3.1 8B Fine-Tuning Playbook

This playbook documents the successful local training of a LoRA adapter on top of **Meta-Llama-3.1-8B-Instruct-4bit** using Apple's MLX Framework on Apple Silicon (M5 Mac).

It is structured to serve as an instant, future-proof guide for replicating, testing, and managing this local corporate triage microservice.

🏗️ System Architecture & Environment

- **Platform:** Apple Silicon (M5 MacBook Pro) with Unified Memory
- **Python Environment:** `.venv` (Python 3.9+)
- **Base Model:** `mlx-community/Meta-Llama-3.1-8B-Instruct-4bit` (~5.5 GB)
- **Target Dataset:** `Lisara/desktriage-dataset` (Hugging Face)
- **Output Adapters:** `./m5_adapters_8b` (~100 MB)

📡 Step 1: Data Preparation (`prepare_data.py`)

Because the Hugging Face dataset `Lisara/desktriage-dataset` already contains pre-formatted Llama 3 syntax structures (including `<|begin_of_text|>`, `<|start_header_id|>`, `<|end_header_id|>`, etc.), we do not need complex Jinja2 token mapping. We simply export the raw text arrays into training and validation JSONL files.

```
# filepath: prepare_data.py
import json
from datasets import load_dataset

print("📡 Downloading dataset from Hugging Face Hub...")
raw_dataset = load_dataset("Lisara/desktriage-dataset")["train"]

def save_mlx_jsonl(split_data, filename):
    with open(filename, "w") as f:
        for row in split_data:
            # The dataset 'text' column already has Llama 3 tags built-in.
            # No Jinja templates or manual dictionary mapping needed.
            f.write(json.dumps({"text": row["text"]}) + "\n")

print("✂️ Splitting data...")
split = raw_dataset.train_test_split(test_size=0.1, seed=42)

save_mlx_jsonl(split["train"], "train.jsonl")
save_mlx_jsonl(split["test"], "valid.jsonl")
print("✅ Done! Raw pre-formatted text saved.")
```

Run this command in the terminal to prepare the data:

```
python prepare_data.py
```

🚀 Step 2: Safe LoRA Fine-Tuning Command

To train an 8 Billion parameter model without causing exploding gradients (which results in `NaN` training loss), we use a stabilized learning rate of $\eta = 5 \times 10^{-5}$ (`5e-5`) instead of the default 2×10^{-4} (`2e-4`).

We omit prompt-masking (`--mask-prompt`) because the dataset is already a flattened text sequence.

```
python -m mlx_lm lora \
  --model mlx-community/Meta-Llama-3.1-8B-Instruct-4bit \
  --data . \
  --train \
  --learning-rate 5e-5 \
  --batch-size 2 \
  --iters 400 \
  --save-every 100 \
  --grad-checkpoint \
  --adapter-path ./m5_adapters_8b
```

🧪 Step 3: Running the Baseline (Un-Tuned) Test

Running a baseline test against the raw, un-tuned 8B model demonstrates why fine-tuning is required. Raw models tend to write markdown blocks or add conversational filler that crashes production code parsers.

```
# filepath: test_baseline_8b.py
from mlx_lm import load, generate

print("📡 Loading the RAW, UN-TUNED Llama-3.1-8B model from cache...")
model, tokenizer = load("mlx-community/Meta-Llama-3.1-8B-Instruct-4bit")

ticket_text = (
    "Hey team, I started my paternity leave tracking log on the HR portal, but th
    "crashed mid-way through. Now my employee dashboard is frozen, I can't log ba
    "my direct deposit details for this month's payroll cycle didn't save correct
)

prompt = f"<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\nYou are

print("\n🚀 Running Baseline Inference on M5 GPU...")
output = generate(model, tokenizer, prompt=prompt, max_tokens=300)

print("\n📩 === [RAW 8B BASELINE MODEL OUTPUT] ===")
print(output)
print("==========================================\n")
```

Run this command to check the raw baseline output:

```
python test_baseline_8b.py
```

🤖 Step 4: Running the Fine-Tuned Test (`test_model.py`)

This script loads the base model and snaps your custom `m5_adapters_8b` on top. It outputs pure JSON, matching your company's specialized schema constraints exactly, starting with `{` and ending with `}` .

```
# filepath: test_model.py
from mlx_lm import load, generate

print("🧠 Loading your custom fine-tuned 8B model and adapters...")
model, tokenizer = load(
    "mlx-community/Meta-Llama-3.1-8B-Instruct-4bit",
    adapter_path="./m5_adapters_8b"
)

ticket_text = (
    "Hey team, I started my paternity leave tracking log on the HR portal, but th
    "crashed mid-way through. Now my employee dashboard is frozen, I can't log ba
    "my direct deposit details for this month's payroll cycle didn't save correct
)

prompt = f"<|begin_of_text|><|start_header_id|>system<|end_header_id|>\n\nYou are

print("\n🚀 Running Fine-Tuned Inference on M5 GPU...")
output = generate(model, tokenizer, prompt=prompt, max_tokens=300)

print("\n🤖 [FINE-TUNED MODEL OUTPUT]:")
print("=" * 40)
print(output)
print("=" * 40)
```

Run your custom tuned pipeline:

```
python test_model.py
```

🧹 Step 5: Storage Maintenance & Space Recovery

Your M5 Mac caches massive base models (like the 5.5 GB Llama 3.1 8B) globally to prevent repeated downloads. You can easily delete or manage these models while preserving your tiny 100 MB fine-tuned adapter folder.

To cleanly view and purge the Hugging Face cache on your system, run:

```
huggingface-cli delete-cache
```

Use the arrow keys to select cached weights (e.g., old 1B models or base 8B models) to delete, freeing up space instantly.

📚 Key Software Engineering Takeaways

1. The "NaN" (Not a Number) Gradient Phenomenon

During machine learning backpropagation, gradients are used to scale step-sizes for weight adjustments. If the learning rate is too high, adjustments grow exponentially until they break the computer's floating-point precision capabilities, resulting in infinite or `NaN` values. Large, quantized models (like 8B 4-bit) require careful step sizes (5×10^{-5}) to maintain stability.

2. Base Models vs. Fine-Tuned Microservices

- **Base Models:** Excellent general-purpose conversationalists, but inherently unpredictable. They wrap output in markdown formatting (```` ```json ````), include conversational filler, and use inconsistent schema patterns.
- **Fine-Tuned Models:** Act as predictable microservices. By training the mathematical weights directly on raw JSON target layouts, the model learns to output strict schema formatting without